VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Quiz Review: Theory

Advanced System Architectures

CO5059

Instructor: Tran Ngoc Thinh

Notes by: Phan Nhat Minh (minh.phannhat95@hcmut.edu.vn)

Semester: 261

Condensed theory from Lectures 1–3 (performance & cost, instruction set architecture, pipelining): the definitions, facts, comparisons and reasons most likely to be asked, followed by a self-check question bank with answers.

Last updated: October 3, 2026

Contents

1 Lecture 1: Computer Architecture, Performance and Cost **1**
- 1.1 What computer architecture is . . . 1
- 1.2 Technology trends and the multicore shift . . . 1
- 1.3 The design cycle . . . 1
- 1.4 Measuring performance . . . 1
- 1.5 MIPS and MFLOPS – why they mislead . . . 2
- 1.6 Benchmarks and summarizing . . . 2
- 1.7 Cost . . . 2

2 Lecture 2: Instruction Set Architecture **3**
- 2.1 Basics . . . 3
- 2.2 ISA styles (where the operands are) . . . 3
- 2.3 Addressing modes . . . 3
- 2.4 Operands, encoding, registers . . . 3
- 2.5 CISC vs. RISC . . . 4
- 2.6 MIPS . . . 4

3 Lecture 3: Pipelining **4**
- 3.1 Concepts . . . 4
- 3.2 The 5-stage MIPS pipeline . . . 5
- 3.3 Hazards . . . 5
- 3.4 Control hazards and branches . . . 5

4 Self-Check Question Bank **6**

5 Answers **7**

1 Lecture 1: Computer Architecture, Performance and Cost

1.1 What computer architecture is

Definition 1.1 (Computer architect). Designs and engineers the levels of a computer system to maximize **performance** and **programmability** within the limits of **technology** and **cost**. Tasks: instruction set design, functional organization, logic design, implementation (IC design, packaging, power, cooling).

- The focus has shifted: arithmetic (1950s–60s) → ISA design (70s–80s) → CPU/memory/I/O/multiprocessors (90s) → multicore, on-chip networks, power (2000s) → low-power/mobile (2010s) → AI chips, quantum (2020s).
- **Levels of abstraction**: software (applications, compiler, assembler, OS, linker/loader, drivers) above the **ISA**, hardware (processor, memory, I/O, datapath & control, logic, circuits, layout) below. The ISA is the hardware/software interface: one ISA, many implementations.
- **Von Neumann / stored-program computer** (EDSAC, 1949): the same memory holds instructions and data – still the basic design today.

1.2 Technology trends and the multicore shift

- **Moore's Law**: transistors per cost-effective chip double every 18–24 months.
- Uniprocessor performance grew ∼25%/yr (VAX era), ∼52%/yr (RISC/x86, 1986–2002), then slowed sharply.
- **Power wall**: power is expensive, transistors are cheap. **ILP wall**: diminishing returns from more hardware for instruction-level parallelism. **Memory wall**: memory is slow (∼200 cycles) relative to arithmetic. Together: the **brick wall**.
- Result: industry moved to **multicore** – many simpler cores are more power-efficient than one complex core ("the processor is the new transistor"). The hard part is software: programming many cores.

1.3 The design cycle

Evaluate existing systems for bottlenecks (**performance**, using *benchmarks*) → simulate new designs (**technology**, using *workloads*) → implement the next generation (**cost**, *implementation complexity*) → repeat.

1.4 Measuring performance

Definition 1.2 (Latency vs. throughput). **Latency** (execution/response time): time to finish one task. **Throughput** (bandwidth): tasks completed per unit time. They often conflict (Concorde has lower latency, the 747 higher throughput).

- **CPU time** excludes I/O wait and other programs; it is user CPU time + system CPU time.
- Performance = 1/ExTime; Speedup = $\text{ExTime}_{old}/\text{ExTime}_{new}$; "X is n% faster than Y": $\text{ExTime}_Y/\text{ExTime}_X = 1 + n/100$.
- **CPU equation**: $T = I \times \text{CPI} \times C$. Program, compiler and ISA affect I and CPI; organization affects CPI and C; technology affects C.

- **Execution time of real programs is the only reliable measure**. MIPS, MFLOPS, clock rate, cache size alone are not.

Theorem 1.3 (Amdahl's Law). Speedup $= 1/\big((1-F)+F/S\big)$: the improvement is limited by the fraction of time F the enhancement can be used; even with $S \to \infty$, speedup $\leq 1/(1-F)$.

Note. **Make the common case fast.** Every instruction is fetched but only some access data ⇒ optimize instruction fetch. Programs show **locality** (90% of time in 10% of code; *temporal*: reuse soon, *spatial*: neighbours soon) ⇒ a memory **hierarchy** (registers, cache, memory, disk) keeps frequent accesses in small fast memories.

1.5 MIPS and MFLOPS – why they mislead

- MIPS $= I/(T \cdot 10^6) = f/(\text{CPI} \cdot 10^6)$ ignores instruction count: (1) depends on the ISA, (2) varies between programs on one machine, (3) can vary *inversely* with performance. Valid only for the same program, ISA and compiler. *Peak* MIPS uses only the lowest-CPI instructions.
- MFLOPS: better across different ISAs, but program-dependent (a compiler has ≈ 0 FP ops) and dependent on the FP operation mix; *peak* MFLOPS is unrepresentative.

1.6 Benchmarks and summarizing

- Benchmark program levels, most to least accurate: **real applications, modified applications, kernels, toy benchmarks, synthetic benchmarks**.
- **SPEC** (System Performance Evaluation Cooperative): SPEC89 (single SPECmark), SPEC92 (int/fp split), SPEC95, SPEC2000, SPEC CPU2006; SPECSFS and SPECWeb for servers.
- Arithmetic mean tracks total time but favours long programs; weighted mean depends on the chosen weights; **geometric mean** of normalized times is consistent whatever the reference machine, but does not track total time; harmonic mean averages rates.
- "Benchmarks shape a field": vendors may tune for benchmarks rather than real programs ("sales almost always wins").

1.7 Cost

- **Cost** = what it takes to build; **price** = what it sells for. List price = component cost + direct cost (labour, scrap, warranty) + gross margin (R&D, marketing, profit) + average discount.
- IC cost is driven by **yield**, **volume** and **feature size**. Larger dies: fewer dies per wafer *and* lower yield, so die cost grows faster than die area.
- Learning curve: cost drops $\approx 10\%$ each time volume doubles. High-end machines sell in low volume, so they need high margins to recoup R&D.

2 Lecture 2: Instruction Set Architecture

2.1 Basics

- **Instruction set**: all instructions a processor can execute; the interface between software and hardware.
- **Execution cycle**: instruction fetch → decode → operand fetch → execute → write back (store result).
- Instruction fields: **opcode**, **source operand reference(s)**, **result operand reference**.
- Design aspects: operation repertoire, addressing modes, data types, instruction format, registers.
- Instruction types: **data processing** (arithmetic, logic), **data storage** (load/store memory), **data movement** (register/I/O), **control** (test, branch).

2.2 ISA styles (where the operands are)

Stack	operands on top of stack; dense code, simple HW; stack bottleneck, little parallelism
Accumulator	one implicit register; few instructions; heavy memory traffic, variable CPI
Memory-memory	all operands in memory; simple but memory-bound
Register-memory	one operand may be in memory (x86, 68000)
Load-store	ALU ops on registers only; `load`/`store` access memory (MIPS, PowerPC, Alpha) – the modern choice

Other styles: high-level-language architectures (Burroughs B5000) failed for lack of compilers able to exploit them; RISC simplifies hardware and lets the compiler build complex operations.

2.3 Addressing modes

Definition 2.1 (Addressing mode). How the processor refers to an operand (or a jump target). Classified by the source of data (immediate, register, memory) and the address calculation (direct, indirect, indexed).

Immediate, register, direct, register indirect, displacement (base+offset), indexed (base+index), memory indirect, PC-relative, scaled. Measurements: **immediate and displacement are by far the most used**; most displacements fit in about 12–16 bits, so 16-bit fields suffice.

2.4 Operands, encoding, registers

- Operand types: characters (ASCII 8-bit, Unicode 16-bit), integers (2's complement, 16/32-bit), floating point (single 32-bit, double 64-bit), BCD (business), fixed point (DSP), RGBA (graphics).
- Encoding: **variable length** (VAX, x86: dense code, hard decode), **fixed length** (MIPS, ARM, PowerPC: easy decode, bigger code), **hybrid** (MIPS16, Thumb: two fixed lengths).
- Registers are the fastest storage and flexible for compilers, but more registers need more encoding bits and a bigger register file can be slower.

2.5 CISC vs. RISC

CISC	RISC
More work per instruction, compact code	Fewer, simpler instructions; more instructions executed
Born when memory was very expensive	Designed for pipelining, CPI ≤ 1
Many addressing modes (68000: 14; 68020: 25; 680x0 listing: 18; IA-32: 12)	Few modes (Alpha: 4; HP PA-RISC: 7)
Variable-length instructions (x86: 1–12 bytes)	Fixed 32-bit instructions
Memory operands in ALU instructions; backward compatibility	Load-store; delayed loads/branches; prefetch and speculation

2.6 MIPS

- 32 general registers, plus PC, Hi, Lo; three 32-bit formats: **R** (op rs rt rd shamt funct), **I** (op rs rt imm16), **J** (op addr26); opcode always in the first 6 bits.
- **Design principles**: simplicity favours regularity; smaller is faster; make the common case fast; good design demands good compromises (three formats, all the same length).
- SPEC2006 mix: integer code is dominated by data transfer (35%) and conditional branches (34%); FP code by arithmetic (48%) and data transfer (36%).
- **Big-endian** (most significant byte at the lowest address).
- MIPS addressing: register, immediate, base+displacement (`lw`/`sw`), PC-relative (`beq`/`bne`, offset in words from PC+4), pseudo-direct (`j`/`jal`), register indirect (`jr`).
- No `MOV`, `NOT`, `SUBI`, `SLA`: use `add …,$zero`, `nor …,$zero`, `addi` with a negative constant, `sll`. 32-bit constants: `lui` + `ori`.
- **Atomic read-modify-write**: `ll`/`sc`; `sc` fails (returns 0) if the word changed since the `ll`.
- **Procedure call**: save return address, save needed registers, execute, restore registers, restore return address, return. `jal` sets `$ra`; `jr $ra` returns. `$s0--$s7, $sp, $fp, $gp, $ra` are preserved across calls (callee saves); `$t`, `$a`, `$v` are not (caller saves if needed). A leaf procedure need not save `$ra`.

3 Lecture 3: Pipelining

3.1 Concepts

Definition 3.1 (Pipelining). Overlapping the execution of multiple instructions. Each step is a **pipeline stage**; stages are connected linearly; the **machine cycle** is set by the slowest stage.

- Increases **throughput**; does **not** reduce (may slightly increase) the **latency** of one instruction (k cycles).
- Ideal CPI = 1: one instruction completes per cycle once the pipeline is full (CPI $= (k+n-1)/n \to 1$).
- Ideal speedup = number of stages. Reduced by unbalanced stages, fill/drain time, latch/control overhead, and hazards.
- **Laundry lessons**: rate limited by the slowest stage; tasks overlap using different resources; dependencies force waiting.

3.2 The 5-stage MIPS pipeline

- IF (instruction memory) – ID (decode, read registers) – EX (ALU, address) – MEM (data memory) – WB (write register). Pipeline registers IF/ID, ID/EX, EX/MEM, MEM/WB carry values and control between stages.
- Register file: **write in the first half, read in the second half** of the cycle.
- **Why MIPS is easy to pipeline**: (1) all instructions the same length, (2) few formats with register fields in fixed positions (read registers while decoding), (3) memory operands only in `lw`/`sw` (EX computes the address, MEM accesses), (4) aligned operands (one access per transfer).

3.3 Hazards

Definition 3.2 (Hazard). A situation that prevents the next instruction from executing in its designated cycle, forcing stalls and raising CPI above 1: **structural** (resource conflict), **data** (dependence on a previous result), **control** (branches and other PC changes).

- CPI_{pipe} = 1+ stall cycles per instruction; speedup = depth/(1+stall CPI) for balanced stages.
- **Structural**: e.g. one memory port for instructions and data. Fix by duplicating resources (separate I/D memories) or stalling; sometimes accepted when rare and costly to remove.
- **Data hazard classes** (I before J): **RAW** (true dependence; J reads too early), **WAW** and **WAR** (name dependences; writes in the wrong order / J writes before I reads); RAR is not a hazard. Only RAW occurs in the simple in-order 5-stage pipeline.
- **Forwarding (bypassing)**: send a result from the EX/MEM or MEM/WB pipeline register straight to the ALU input, instead of waiting for write-back.
- **Load-use hazard**: a load's data is ready only after MEM, so the next instruction still stalls one cycle even with forwarding.
- **Hardware interlock**: detects the hazard and stalls. **Compiler (static) scheduling**: reorders code so a load is not immediately followed by its use. Static = compile time, dynamic = run time in hardware.

3.4 Control hazards and branches

- Branch penalty = (stage where the branch is resolved) -1: 3 cycles if resolved in MEM. MIPS moves the zero test and target adder into ID $\Rightarrow$ 1 cycle.
- **Four alternatives**: (1) stall until resolved; (2) **predict not taken** – keep fetching, squash if taken (late state update makes this safe; $\approx$ 47% of MIPS branches not taken); (3) **predict taken** ($\approx$ 53% taken) – no gain in MIPS because the target is not known earlier; (4) **delayed branch** – the instruction in the delay slot always executes (MIPS uses one slot).
- Delay-slot filling: from **before** the branch (best, always useful, branch must not depend on it); from the **target** (useful if taken, may need copying, must be safe if not taken); from the **fall-through** (useful if not taken, must be safe if taken). **Cancelling (nullifying) branches** make the last two easier.
- Compilers fill $\approx$ 60% of slots, $\approx$ 80% of them usefully ($\approx$ 50% overall). Delayed branches lost popularity: deeper and multiple-issue pipelines need more slots, and dynamic prediction became cheap.

4 Self-Check Question Bank

Answers are in the next section.

Q1. What is the role of a computer architect?

Q2. Why is the ISA called the hardware/software interface?

Q3. Name the three "walls" and the conclusion drawn from them.

Q4. Latency vs. throughput: which one does pipelining improve?

Q5. Which of I, CPI, C does (a) the compiler, (b) the organization, (c) the technology affect?

Q6. State Amdahl's Law and its main consequence.

Q7. Give three reasons why MIPS is a poor performance metric. When is it valid?

Q8. Order the five kinds of benchmark programs from most to least accurate.

Q9. Why use the geometric mean of normalized times? What is its weakness?

Q10. Why does die cost grow faster than die area?

Q11. Cost vs. price: what is between them?

Q12. List the five steps of the instruction execution cycle.

Q13. Compare stack and accumulator ISAs (one advantage and one drawback each).

Q14. Which addressing modes are used most, and what does that imply for instruction design?

Q15. Fixed vs. variable-length encoding: trade-off?

Q16. Give four characteristics of RISC.

Q17. State the four MIPS design principles.

Q18. How do you perform `NOT`, `MOV`, and load a 32-bit constant in MIPS?

Q19. What do `ll`/`sc` provide?

Q20. Which registers must a callee preserve? Must a leaf procedure save `$ra`?

Q21. Why is the ideal CPI of a pipeline 1 even though each instruction takes 5 cycles?

Q22. What limits pipeline speedup in practice?

Q23. Give four reasons why MIPS is easy to pipeline.

Q24. Define the three kinds of hazard with an example of each.

Q25. Why can't WAR and WAW hazards occur in the 5-stage MIPS pipeline?

Q26. What is forwarding, and which hazard can it not remove?

Q27. How does MIPS reduce the branch penalty from 3 to 1 cycle?

Q28. Compare predict-not-taken and delayed branch.

Q29. What are the three ways to fill a delay slot and their conditions?

Q30. Why have delayed branches fallen out of favour?

5 Answers

A1. Design the levels of a computer system to maximize performance and programmability within technology and cost limits.

A2. It is the only layer both sides agree on: software is written to it and every hardware implementation must execute it, so one ISA supports many implementations.

A3. Power wall, ILP wall, memory wall (together the brick wall) ⇒ single-core performance growth slowed, so chips went multicore.

A4. Pipelining improves throughput, not latency.

A5. (a) I and CPI; (b) CPI and C; (c) C.

A6. Speedup $= 1/((1-F)+F/S)$; the part not improved limits the overall gain (at most $1/(1-F)$) ⇒ make the common case fast.

A7. ISA-dependent; varies between programs; can vary inversely with performance (ignores instruction count). Valid only with the same program, ISA and compiler.

A8. Real applications, modified applications, kernels, toy benchmarks, synthetic benchmarks.

A9. Its ranking does not depend on the reference machine. It does not reflect total execution time.

A10. A larger die means fewer dies per wafer and a lower yield (more likely to contain a defect), so cost rises faster than area.

A11. Direct costs, gross margin and the average discount are added to the component cost to give the list price.

A12. Fetch, decode, operand fetch, execute, write back.

A13. Stack: dense code / stack bottleneck. Accumulator: few instructions, easy compiler / heavy memory traffic, variable CPI.

A14. Immediate and displacement; so a good ISA supports them efficiently with ≈ 16-bit fields.

A15. Fixed: simple, fast decode (good for pipelining) but larger code; variable: compact code but complex decode.

A16. Simple instructions, fixed-length encoding, load-store, few addressing modes, designed for pipelining with CPI ≈ 1 (also delayed branches/loads).

A17. Simplicity favours regularity; smaller is faster; make the common case fast; good design demands good compromises.

A18. `nor $d,$s,$zero`; `add $d,$s,$zero`; `lui` upper half then `ori` lower half.

A19. An atomic read-modify-write: `sc` succeeds only if the word was not modified since the `ll`.

A20. `$s0--$s7`, `$sp`, `$fp`, `$gp`, `$ra`. No – a leaf never executes `jal`, so `$ra` is untouched.

A21. Once full, one instruction finishes every cycle; the $k-1$ fill cycles are paid once, so CPI $= (k+n-1)/n \to 1$.

A22. Unbalanced stages, pipeline-register overhead, fill/drain time, and hazards (stalls).

A23. Same instruction length; few formats with fixed register fields; memory operands only in loads/stores; aligned operands.

A24. Structural (one memory port for instruction and data accesses), data (`add r1`; `sub` using `r1`), control (a taken `beq`).

A25. Reads happen early (ID) and writes late (WB), and instructions stay in order, so a later instruction can neither write before an earlier one reads nor before an earlier one writes.

A26. Routing a result from a pipeline register directly to the ALU input of a later instruction. It cannot remove the load-use stall.

A27. It tests the register in ID and computes the target with an extra adder in ID, so the branch is resolved in stage 2.

A28. Predict-not-taken fetches sequentially and squashes on a taken branch (penalty only when taken); delayed branch always executes the slot instruction (cost only when the slot holds no useful work).

A29. From before the branch (independent of the branch); from the target (safe if not taken, may duplicate code); from the fall-through (safe if taken).

A30. Deeper and multiple-issue pipelines need more than one slot, and dynamic branch prediction became cheap.